HESIA Alpha Architecture Memo

Date: 2026-05-09

Objective

Alpha targets a compact multimodal embedded AI stack for drone autonomy under three constraints:

- BitNet-style ternary BitLinear projections for memory and compute reduction.
- Mamba-2-inspired temporal recurrence with linear sequence-time inference.
- Lottery-ticket pruning to identify a smaller deployable subnet after dense pretraining.

Current Implementation State

Implemented in this phase:

- ml/hesia_m2b/bitlinear.py: added bitlinear_parameter_report for BitLinear coverage and ternary storage estimates.
- ml/hesia_m2b/model.py: converted the M2B SSM projections, visual/state projections, and risk head to BitLinear.
- ml/hesia_m2b/mission_model.py: converted mission visual/state projections, risk head, stage head, and instance class head to BitLinear.
- ml/hesia_m2b/infer.py: added a JSON-emitting inference entry point for checkpoint, RGB/depth sequence, state vector, mission id, action, risk, and heatmap shape.
- ml/hesia_m2b/hf_alpha_manifest.py: added a machine-readable Hugging Face source manifest for DINOv3 teachers and drone datasets.
- artifacts/alpha/hf_alpha_manifest.json: generated manifest.

Architecture

The candidate architecture keeps the existing HESIA M2B shape because it already matches embedded constraints:

1. RGB and depth encoders produce compact spatial features.
2. A fused visual token is combined with telemetry state and mission embedding.
3. A stack of export-friendly selective SSM blocks processes the temporal sequence.
4. Heads emit action chunks, risk score, and dense heatmaps.
5. Mission branch extends the base model with semantic logits, instance masks, instance classes, and stage logits.

The important change is that all main projection layers in the temporal and decision path now use BitLinear. Convolutions remain dense because the current BitNet constraint was scoped to linear layers and because quantized convolution export requires separate calibration.

DINOv3 Strategy

DINOv3 is not embedded directly in the deployed model. Instead, it is used as a teacher:

- Teacher candidates: facebook/dinov3-vits16-pretrain-lvd1689m and facebook/dinov3-convnext-tiny-pretrain-lvd1689m.
- Student: HESIA compact visual encoder.
- Training objective: supervised mission losses plus feature distillation from teacher dense/global embeddings.
- Deployment: only the compact student is exported to ONNX/TensorRT.

This avoids putting a heavy foundation model on Jetson while still benefiting from DINOv3 visual features during training.

Temporal Strategy

SelectiveSSMLite remains a portable Mamba-2-inspired block:

- It maintains a recurrent state over the sequence.
- It avoids quadratic attention.
- It uses a loop over sequence length, so inference cost grows linearly with sequence length.
- It is written in PyTorch primitives for ONNX export rather than depending on custom CUDA kernels.

It is not claimed to be a byte-for-byte Mamba-2 implementation.

Lottery-Ticket Strategy

The existing ticket.py implements iterative magnitude pruning:

- Start dense.
- Capture rewind state.
- Prune a global fraction of alive weights.
- Rewind surviving weights and retrain.
- Export masks and density statistics.

Next training should use the same flow, then compare dense vs ticket checkpoint with identical benchmark scripts on RTX 3070.

Hugging Face Training Sources

The generated manifest contains:

- DINOv3 teacher models.
- DroneScapes and DroneScapes2 for UAV multimodal visual data.
- Annotated DroneScapes2 split for fast segmentation sanity checks.
- Drone detection datasets for object/head robustness.

Every source still needs license review before redistributable checkpoints are published.

Training And Benchmark Addendum

Executable Alpha work was completed after the initial memo:

- Synthetic CUDA ticket training produced artifacts/alpha/hesia_alpha_synthetic_ticket.pt.
- Hugging Face image sampling downloaded 192 training images from Voxel51/dronescapes2_annotated_train_set and pathikg/drone-detection-dataset.
- DINOv3 direct teacher loading was attempted with facebook/dinov3-vits16-pretrain-lvd1689m, but Hugging Face returned a gated-repo 401 because this machine is not authenticated for that model.
- The executable fallback teacher was facebook/dinov2-small.
- Vision distillation produced artifacts/alpha/hesia_alpha_distilled.pt.
- CUDA inference, ONNX export, ONNX checker, and benchmark runs completed.

Final compact checkpoint evidence:

Metric	Value
Parameters	706,067
BitLinear layers	31
Dense Linear layers	0
Ticket sparsity	32.5%
Final checkpoint size	3.62 MB
Final ONNX size	2.99 MB
HESIA full forward on RTX 3070	11.12 ms
DINOv2-small teacher feature extraction on RTX 3070	14.81 ms
Final held-out distillation cosine	0.4451

The model is now a real functional prototype, but two Alpha ambitions are still not honestly met: DINOv3-level vision cannot be claimed without gated-model access, and market-comparable verbal behavior is absent because no trained language decoder is present.

Verification Performed

- python -m py_compile ml\hesia_m2b\bitlinear.py ml\hesia_m2b\model.py ml\hesia_m2b\mission_model.py ml\hesia_m2b\hf_alpha_manifest.py ml\hesia_m2b\infer.py
- python ml\hesia_m2b\hf_alpha_manifest.py

Later CUDA verification was run with PyTorch 2.11.0+cu128 on RTX 3070. See artifacts/alpha/HESIA_ALPHA_TRAINING_BENCHMARK_REPORT.md for the training logs, benchmark table, and blocker list.

Next Executable Phase

1. Install or activate the PyTorch CUDA environment.
2. Run a CPU smoke inference with infer.py and zero-filled inputs.
3. Run a short synthetic training on RTX 3070.
4. Run dense benchmark and ticket-pruned benchmark with the same sequence length and batch size.
5. Add DINOv3 distillation loader after license and storage checks.
6. Export ONNX and verify TensorRT path.